NEXORA Cloud Storage Platform

System Architecture, Requirements, Algorithms & Parameter Specification

IN Made in India · Built for Bharat · Zero-Knowledge Architecture

Document: Technical Specification
Version: 1.0.0 (Production)
Date: September 30, 2026
Security: Client-Side AES-256-GCM

1 Software & Hardware Requirements

1.1 Environment & Runtime Specifications

Component	Minimum Requirement	Recommended Specification
Operating System	Windows 10, macOS 12 Monterey, Ubuntu 20.04 LTS	Windows 11 / macOS Sonoma / Ubuntu 22.04 LTS (x64 / ARM64)
Node.js Runtime	Node.js v18.17.0+	Node.js v20.x or v22.x LTS
Package Manager	npm v9.0.0+ / yarn 1.22+ / pnpm 8+	npm v10+ (Included with Node.js)
Web Browser	Chrome 95+, Microsoft Edge 95+, Safari 15.4+, Firefox 102+	Modern Chromium Browser (Chrome / Edge with WebCrypto & Streams)
Client APIs	Web Crypto API, Fetch API, Blob, FileReader	Full CompressionStream & Subresource Integrity Support

1.2 Client & Host Hardware Requirements

Hardware Resource	Minimum Allocation	Optimal Operational Setup
Processor (CPU)	Dual-Core 2.0 GHz (x86_64 or Apple Silicon M1)	Quad-Core 2.8 GHz+ (for parallel chunk hashing & compression)
Memory (RAM)	4 GB System Memory	8 GB to 16 GB (for concurrent multi-part client array buffers)
Storage (Disk)	500 MB free disk space for bundle & cache	2 GB+ SSD for local builds, static traces, and database staging
Network Uplink	1.5 Mbps upload bandwidth	15 Mbps+ Broadband for parallel multi-part chunk uploads

1.3 Cloud & Database Infrastructure

> **PostgreSQL Database & S3 Storage:** Configured via Supabase Cloud. Contains relational tables (`profiles`, `files`, `activity_logs`, `notifications`) and binary object bucket (`nexora-files`) with row-level policies.

2 Tools and Technologies Used

Layer / Domain	Technology & Version	Role in System Architecture
Full-Stack Framework	`Next.js 15.5.26` (App Router)	Hybrid SSR/SSG rendering, route handling, dynamic parameters, and server client routing.
Frontend Library	`React 19.0.0`	Component lifecycle, concurrent rendering, and reactive state stores.
Type System	`TypeScript 5.7.3`	Strict static typing for file schemas, cryptographic tokens, and API communication.
Styling & Design System	`Tailwind CSS 3.4.17`	Utility-first responsive styles, glassmorphism, glowing ambient lighting, and Indian SaaS design.
Database & Real-Time	`Supabase PostgreSQL`	Primary relational storage for file metadata, user stats, aggregate metrics, and storage usage.
Cloud Object Store	`Supabase Storage`	S3-compatible persistent storage bucket (`nexora-files`) for uploaded file binaries.
Authentication	`Supabase Auth SDK`	Zero-knowledge auth tokens, password recovery links, and user session management.
Cryptography	`W3C Web Crypto API`	Client-side SHA-256 fingerprinting and AES-256-GCM symmetric buffer encryption.
Stream Compression	`CompressionStream API`	Lossless real-time Gzip stream deflate compression on client before network dispatch.
Icons & Micro-UI	`Lucide React & Framer Motion`	Consistent visual iconography and smooth UI micro-transitions.

3 Project Simulation & Algorithm Implementation

NEXORA features an end-to-end **9-Step Zero-Knowledge Client-Side Cryptographic Pipeline** executed whenever a user uploads an asset:

Step 0: File Intake
Header validation, MIME type detection, byte-size calculation.

Step 1: SHA-256 Digest
High-speed cryptographic hash generated via WebCrypto API.

Step 2: Deduplication
Content-addressable check; eliminates redundant cloud storage.

Step 3: Compression
Lossless Gzip stream compression; reduces payload size by ~60%.

Step 4: AES-256 Encryption
Military-grade AES-256-GCM authenticated cipher with 96-bit IV.

Step 5: Chunk Partitioning
Payload sliced into 8 discrete parts with individual sub-hashes.

Step 6: Parallel Upload
Multi-part streaming to Supabase Storage bucket.

Step 7: Verification
Server-side checksum confirmation and tamper verification.

Step 8: Vault Committed
Row inserted to Supabase PostgreSQL; user storage updated.

Algorithm 1: Cryptographic Fingerprinting (SHA-256)

Generates a deterministic 64-character hexadecimal digest representing the file's binary stream:

```
Digest = HexEncode( SHA-256( ArrayBuffer(File) ) )
```

Implemented in `lib/crypto-engine.ts` using `crypto.subtle.digest("SHA-256", buffer)`.

Algorithm 2: Content-Addressable Deduplication (CAD)

Before transmitting file bytes to cloud storage, the system checks if the file digest `H_new` already exists in the user's database:

```
IF ∃ H_existing = H_new THEN Duplicate = TRUE → Link Metadata → Bypass Binary Upload
```

This zero-knowledge deduplication saves user bandwidth and instantly credits duplicate savings in user analytics.

Algorithm 3: Lossless Stream Compression (Gzip Deflate)

Applies the DEFLATE algorithm through the browser's `CompressionStream("gzip")`:

```
Savings Percentage = ((Original Bytes - Compressed Bytes) / Original Bytes) × 100
```

Algorithm 4: Authenticated Zero-Knowledge Encryption (AES-256-GCM)

Encrypts the compressed payload on the client side prior to network transmission:

```
Ciphertext = AES-GCM-256( Key_256, Nonce_96, Payload_Buffer )
```

The encryption key is generated client-side and never exposed to intermediate transit nodes in plaintext.

Algorithm 5: Dynamic Multi-Part Chunk Partitioning

Partitions the encrypted buffer into `k = 8` uniform blocks:

```
Chunk_Size = ⌈ Total_Bytes / 8 ⌉, Chunk_i = Slice( i × Chunk_Size, min((i+1) × Chunk_Size, Total_Bytes) )
```

4 System Parameter Settings & Configuration Constants

Parameter Name	Configuration Value	Module / Location	Architectural Purpose
AES Symmetric Key Size	`256 bits` (32 bytes)	`lib/crypto-engine.ts`	Zero-knowledge symmetric data encryption standard.
GCM Initialization Vector	`96 bits` (12 bytes)	`lib/crypto-engine.ts`	NIST recommended nonce length for AES-GCM.
Digest Hash Algorithm	`SHA-256`	`lib/crypto-engine.ts`	Collision-resistant content addressable hashing.
Default Multi-Part Chunks	`8 Chunks`	`lib/crypto-engine.ts`	Optimal parallelization for client-to-cloud streams.
Default Free Storage Quota	`100 GB` (107,374,182,400 B)	`lib/storage-store.ts`	Baseline cloud storage allocated to every free user.
Pro Plan Storage Quota	`2,000 GB` (2 TB)	`app/page.tsx`	Expanded capacity for enterprise and pro users.
Minimum Password Length	`6 Characters`	`app/auth/page.tsx`	Enforces authentication credential minimum policy.
Storage Bucket Name	`nexora-files`	`lib/supabase-service.ts`	Target S3-compatible Supabase storage bucket.
Pipeline Visual Delays	`300ms - 650ms`	`upload-pipeline-flow.tsx`	Paced UI simulation of cryptographic processing steps.
Synthetic Compression Ratio	`61.6%`	`lib/crypto-engine.ts`	Fallback savings percentage for unstreamable formats.
Currency Localization	`INR (₹)`	`app/page.tsx`	India-first localization for billing and pricing plans.